“Smart Resource Allocation Using Banker’s Algorithm”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

ILI HARSHIKA	**2520030607**
GURRAPU NIKSHITHA	**2520030512**
ANGELA JOSE	**2520030365**

Under the Esteemed Guidance of

Dr. D. Bheekya

Faculty, Operating Systems and Systems Programming

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **"Smart Resource Allocation Using Banker's Algorithm"** is a record of Bonafide work of **ILI HARSHIKA – 2520030607, GURRAPU NIKSHITHA – 2520030512, ANGELA JOSE – 2520030365** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

ILI HARSHIKA	**2520030607**
GURRAPU NIKSHITHA	**2520030512**
ANGELA JOSE	**2520030365**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project based report entitled **"Smart Resource Allocation Using Banker's Algorithm"** is a bonafide work done and submitted by **Ili Harshika – 2520030607, Gurrapu Nikshitha – 2520030512, Angela Jose – 2520030365** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish is to express our sincere thanks to all those who have assisted us in one way or the other for the completion of our project.

Our greatest appreciation goes to our Course Coordinator **Dr. D. Bheekya** and our guide **Dr. D. Bheekya**, Department of Computer Science and Engineering, whose tremendous support, encouragement and guidance for this project cannot be expressed in words.

We express our gratitude to **Dr. P Venkateshwara Rao (CSE)**, Head of the ***Department of Computer Science Engineering***, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff, and also those who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

ILI HARSHIKA	**2520030607**
GURRAPU NIKSHITHA	**2520030512**
ANGELA JOSE	**2520030365**

Abstract

Modern operating systems run many processes at the same time, and these processes compete for a limited set of resources such as memory blocks, files, printers and other devices. When requests for these resources are not regulated, processes can end up in a circular wait, each holding a resource that the next one needs, and the affected processes are blocked indefinitely. This situation is called deadlock, and it reduces the throughput of the affected threads to zero. To address this problem, our project presents Smart Resource Allocation Using Banker's Algorithm, an implementation of Dijkstra's deadlock-avoidance strategy in the C programming language.

The system represents the resource state with four structures: the Available vector and the Max, Allocation and Need matrices, where Need = Max − Allocation. It is organised into two cooperating parts. The interface module (main.c) reads a scenario and prints the state, and the Banker core (banker.c) implements the Safety Algorithm, which searches for a sequence in which every process can finish, and the Resource-Request Algorithm, which makes a trial allocation, checks the resulting state and rolls it back if it is unsafe. The safety check runs in $O(m \times n^2)$ time for n processes and m resource types.

The system was compiled with GCC using the -Wall and -Wextra flags and tested on Ubuntu Linux. For the standard example of five processes and three resource types with the Available vector (3, 3, 2), it produced the safe sequence P1 → P3 → P4 → P0 → P2. A request was granted only when the resulting state remained safe, a request that would lead to an unsafe state was denied and rolled back, and a deadlock-prone state with a circular wait was correctly reported as unsafe. All seven planned test cases passed. In summary, the project gives a transparent, working demonstration of safe resource allocation and connects Operating System theory on deadlocks with practical programming.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		**4**
	Abstract		**5**
1	**Introduction**		**8–9**
		1.1 Background of the Project	8
		1.2 Problem Statement	8
		1.3 Scope of the Project	8
		1.4 Objectives of the Project	9
		1.5 Importance of the Application	9
		1.6 Target Users / Audience	9
2	**OS Concepts and Background Study**		**10–12**
		2.1 Operating System Concepts Related to the Project	10
		2.2 Existing Methods / Algorithms	11
		2.3 Proposed Approach	11
		2.4 Advantages of the Proposed Approach	12
3	**System Requirements**		**13**
		3.1 Hardware Requirements	13
		3.2 Software Requirements	13
		3.3 Operating System Environment – Ubuntu / WSL	13
		3.4 Development Tools and Compilers	13
4	**Technology and Tools Used**		**14–15**
		4.1 C / C++ Programming	14
		4.2 GCC Compiler	14
		4.3 Linux System Calls	14
		4.4 POSIX Threads / Synchronization	14
		4.5 Linux Commands and Utilities	14
		4.6 Other Tools / Libraries Used	15
5	**System Architecture and Design**		**16–19**
		5.1 Overall System Architecture	16
		5.2 System Workflow	16
		5.3 Flowchart / Data Flow	17
		5.4 Module Description	18

S. No.	Chapter	Topics / Subtopics	Page No.
		5.5 Input and Output Design	19
6	**Implementation**		**20–25**
		6.1 Module-wise Implementation	20
		6.2 Algorithm / Pseudocode	20
		6.3 Core C / C++ Implementation	21
		6.4 Linux System Calls / APIs Used	24
		6.5 Process / Thread Implementation	25
		6.6 IPC / Synchronization Implementation	25
		6.7 Memory Management Implementation	25
		6.8 Optional User Interface Implementation	25
7	**Execution and Demonstration**		**26–30**
		7.1 Compilation in Ubuntu	26
		7.2 Execution Commands	26
		7.3 Sample Inputs and Outputs	26
		7.4 Screenshots of Execution	29
		7.5 Demonstration of OS Concepts	30
8	**Conclusion**		**31**
		8.1 Summary of the Project	31
		8.2 Objectives Achieved	31
		8.3 Results Achieved	31
		8.4 Skills and Learning Outcomes	31
9	**References**		**32**
		9.1 Books	32
		9.2 Research Papers	32
		9.3 Linux / GNU Documentation	32
		9.4 Online Resources	32

CHAPTER - 1 INTRODUCTION

1.1 Background of the Project

An Operating System (OS) is the layer of software that manages the hardware of a computer and shares it among the programs that run on it. Besides the processor, the resources that the OS must manage include memory, files, printers, disks and other input/output devices. Many of these resources can be used by only one process at a time, so a process must request a resource, use it and then release it. When several processes run concurrently, these requests have to be managed carefully.

A deadlock occurs when a set of processes is blocked because each process holds at least one resource and waits for a resource held by another process in the set. Four conditions must hold together for a deadlock to arise: mutual exclusion, hold and wait, no preemption and circular wait. Once a deadlock has occurred, none of the processes involved can proceed, and recovery usually requires outside intervention.

Edsger W. Dijkstra proposed the Banker's Algorithm in the mid-1960s as a way of avoiding deadlock. It takes its name from banking: a banker never lends cash in such a way that he can no longer satisfy the needs of all his customers. In the same way, the operating system grants a resource request only if, after granting it, there is still an order in which every process can obtain what it needs and finish. This project, Smart Resource Allocation, implements that idea in a small and readable program.

1.2 Problem Statement

In a multiprogramming system, processes request resources at unpredictable moments. If these requests are granted without any check, a closed chain of processes can form in which each process holds at least one resource needed by the next, so that all of them wait forever. The affected tasks are suspended indefinitely, and in high-concurrency environments the cost of manual recovery exceeds the benefit of granting every request immediately.

The problem addressed by this project is to develop a system that decides, for every resource request, whether the request can be granted without moving the system into an unsafe state. The system must compute the remaining need of each process, search for a safe sequence using the Safety Algorithm, grant a request only if a safe sequence exists afterwards, and otherwise make the requesting process wait.

1.3 Scope of the Project

The scope of the project covers the following:

- Representation of the system state using the Available vector and the Max, Allocation and Need matrices for up to 10 processes and 10 resource types.
- Calculation of the Need matrix using Need = Max − Allocation.
- Implementation of the Safety Algorithm, which finds a safe sequence or reports an unsafe state.
- Implementation of the Resource-Request Algorithm with trial allocation and rollback.
- Display of the state, the step-by-step safety check and the final decision for each request.
- Implementation and testing on Ubuntu Linux using the GCC compiler.

The project is a user-space simulation of a resource manager. It does not modify the Linux kernel, does not handle processes that arrive or leave dynamically, and assumes that every process declares its maximum demand in advance. A graphical interface, deadlock detection with recovery, and integration with real Linux processes are outside the present scope and are listed as possible future work.

1.4 Objectives of the Project

- To represent the resource-allocation state of a system using the Available, Max, Allocation and Need data structures.
- To compute the Need matrix of all processes from the Max and Allocation matrices.
- To implement the Safety Algorithm and use it to find a safe sequence of process execution.
- To implement the Resource-Request Algorithm so that a request is granted only when the resulting state is safe.
- To demonstrate the algorithm on a standard five-process example and on a deadlock-prone state, and to verify the results with test cases.

1.5 Importance of the Application

The application is important for both academic and practical reasons. Academically, it turns an abstract algorithm from the Operating Systems syllabus into a program whose behaviour can be observed step by step: the printed Work vector shows how resources are released as processes finish, the safe sequence shows the order of execution, and the decision message shows why a request was granted, delayed or denied. Practically, the idea of checking a request against the worst-case future demand of every process is useful wherever a shared pool of resources must be managed without risking deadlock, for example in resource schedulers and in real-time systems where recovery from a deadlock is expensive.

1.6 Target Users / Audience

- Undergraduate students learning Operating Systems and deadlock handling.
- Faculty members who want a compact, demonstrable example of deadlock avoidance for classroom use.
- Beginners in systems programming who want to understand how a resource manager can keep a system in a safe state.

CHAPTER - 2 OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

Resources and Resource Types

A resource is anything a process must obtain before it can continue, such as CPU time, memory, files or I/O devices. Resources are grouped into types, and each type may have several identical instances; for example, a system may have 10 instances of resource type A, 5 of type B and 7 of type C. A process uses a resource in three steps: request, use and release. If a request cannot be granted immediately, the process waits.

Deadlock and the Four Necessary Conditions

A set of processes is deadlocked when each process in the set waits for an event that only another process in the set can cause. Four conditions must hold at the same time: (1) mutual exclusion – at least one resource cannot be shared; (2) hold and wait – a process holds one resource while waiting for another; (3) no preemption – a resource can be released only voluntarily by the process holding it; and (4) circular wait – there is a closed chain of processes in which each waits for a resource held by the next. Avoiding deadlock means making sure that these four conditions can never all be true together.

Resource Allocation Graph and Circular Wait

A resource-allocation graph shows processes, resources and the edges between them. An edge from a resource to a process means that the resource is assigned to the process, and an edge from a process to a resource means that the process is requesting it. Figure 2.1 shows the smallest circular wait: Process A holds RES 1 and requests RES 2, while Process B holds RES 2 and requests RES 1. Neither can proceed, so both are blocked indefinitely.

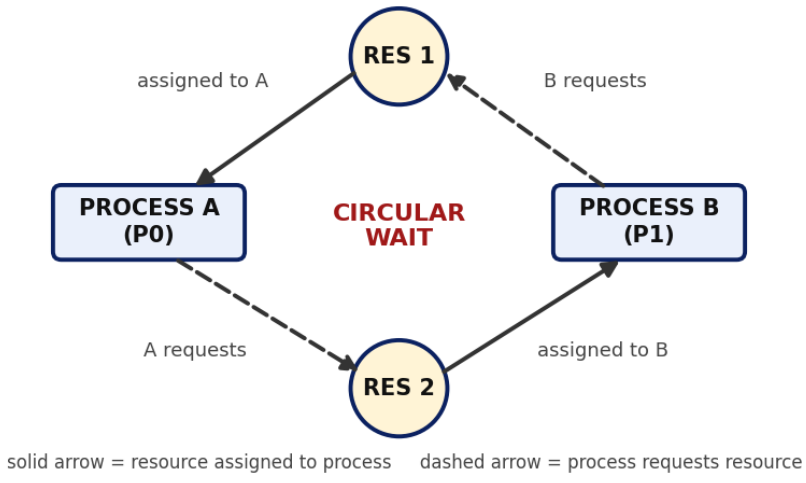


Figure 2.1: Circular wait between two processes and two resources

Safe and Unsafe States

A state is safe if there is at least one order, called a safe sequence, in which every process can be given the resources it still needs and run to completion. If no such order exists, the state is unsafe. A safe state can never lead to a deadlock, whereas an unsafe state may lead to one, although it does not always do so. The Banker's Algorithm therefore keeps the system in safe states only.

Data Structures of the Banker's Algorithm

For n processes and m resource types, the algorithm uses the following structures. The Need matrix is the primary input of the safety simulation.

Structure	Size	Meaning
Available	m	Number of free instances of each resource type
Max	n × m	Max[i][j] is the maximum demand of process Pi for resource Rj
Allocation	n × m	Allocation[i][j] is the number of instances of Rj currently held by Pi
Need	n × m	Need[i][j] = Max[i][j] − Allocation[i][j], the remaining requirement of Pi

2.2 Existing Methods / Algorithms

There are four classical ways of dealing with deadlock in an operating system:

- **Deadlock prevention:** ensures that at least one of the four necessary conditions can never hold, for example by requiring a process to request all its resources at once or by imposing an order on the resource types. It is safe, but it often leads to low resource utilisation.
- **Deadlock avoidance:** the system examines each request and grants it only if the resulting state is safe. It needs advance information about the maximum demand of each process. The Banker's Algorithm and the resource-allocation-graph algorithm belong to this class.
- **Deadlock detection and recovery:** requests are granted freely, the system periodically looks for a deadlock, and if one is found it recovers by terminating processes or preempting resources. It does not restrict requests, but recovery is costly.
- **Ignoring the problem:** the approach taken by most general-purpose operating systems for ordinary resources, which assume that deadlocks are rare and leave it to the user to restart a blocked program.

2.3 Proposed Approach

The project adopts deadlock avoidance using Dijkstra's Banker's Algorithm, because it stops a deadlock from occurring instead of repairing it afterwards. The approach is split into two parts to keep the code clear: the interface module reads the scenario and presents the results, and the Banker core calculates the Need matrix and carries out the Safety Algorithm and the Resource-Request Algorithm. The algorithm works in four steps:

1. State evaluation – analyse the current Available, Max and Allocation matrices to define the baseline of the system.
2. Trial allocation – when a request arrives, create a temporary state in which the request is assumed to be granted.
3. Safety search – run the Safety Algorithm on the temporary state to look for a sequence in which every process can finish.
4. Final decision – grant the request only if a safe sequence exists; otherwise restore the old state and make the requesting process wait.

The Safety Algorithm starts with Work equal to Available and Finish[i] false for every process. It repeatedly finds a process Pi that has not finished and whose Need is less than or equal to Work, assumes that Pi finishes and returns its allocation (Work = Work + Allocation[i]), and marks Finish[i] true. If every process can be marked in this way, the state is safe. Over n processes, each step compares m values, which gives a running time of $O(m \times n^2)$.

As a result, deadlock is avoided: a transition to an unsafe state is denied even when the requested resources are currently idle.

2.4 Advantages of the Proposed Approach

- **Proactive** – deadlock is avoided before it occurs, so no recovery action such as terminating processes is needed.
- **High utilisation** – unlike prevention, a request is refused only when it actually threatens safety, so resources are used as fully as safety permits.
- **Predictable** – every decision follows a clear rule, and the safe sequence shows the order in which the processes can finish.
- **Simple data structures** – only one vector and three matrices are needed, and the check runs in polynomial time, $O(m \times n^2)$.
- **Light-weight** – the project needs only a C compiler and a Linux environment; no external libraries are required.

Limitations. The algorithm requires every process to declare its maximum demand in advance, assumes a fixed number of processes and resources, and can be conservative, because an unsafe state is not always a deadlock. These points are noted as future scope.

CHAPTER - 3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The program is very light-weight, so any modern computer is sufficient. The recommended minimum configuration is listed below.

Component	Requirement
Processor	Any modern 64-bit processor (Intel Core i3 / AMD Ryzen 3 or better)
RAM	4 GB minimum (8 GB recommended when running Ubuntu under WSL)
Storage	At least 2 GB of free disk space for the Linux environment, compiler and project files
Display / Input	Standard monitor, keyboard and mouse

3.2 Software Requirements

Software	Purpose
Ubuntu Linux (native or on WSL)	Operating system on which the project is compiled and executed
GCC (GNU Compiler Collection)	Compiles the C source files into the banker executable
GNU nano / VS Code	Editors used for writing the source code
GDB and terminal	Debugging, running and testing the program

3.3 Operating System Environment – Ubuntu / WSL

The project is a standard C program that uses only the C library, so it is portable, but it was developed and executed in an Ubuntu environment using the standard terminal. On a Windows computer, the Windows Subsystem for Linux (WSL) provides the same environment and lets the code compile and run without any change. The project files are kept in the directory SmartResourceAllocation, which contains the header banker.h, the source files banker.c and main.c, and two sample input files.

3.4 Development Tools and Compilers

The C source code is compiled with GCC. The warning options -Wall and -Wextra are used so that the compiler reports suspicious constructs such as unused variables or missing return values. The two source files are compiled and linked in a single command, producing the executable named banker. GDB can be used to step through the safety algorithm, and the terminal is used for running the tests by redirecting the sample input files to the program.

CHAPTER - 4 TECHNOLOGY AND TOOLS USED

4.1 C / C++ Programming

C is a natural language for systems programming on Linux, because it gives direct control over data layout and uses very little overhead. Matrices and vectors map directly onto C arrays, and a structure can hold the whole system state. The project is written in C, and the header file banker.h defines the State structure and the interface of the Banker core, so that main.c and banker.c can be compiled separately.

4.2 GCC Compiler

GCC translates the C source into a native executable. The command used is:

```
gcc -Wall -Wextra main.c banker.c -o banker
```

Here main.c and banker.c are the two source files, -o names the output file and -Wall -Wextra turn on additional warnings. The header file banker.h is found automatically because it is in the same directory.

4.3 Linux System Calls

The Banker's Algorithm is implemented as a user-space simulation, so the program does not call process-related system calls directly. It uses the standard I/O library, which in turn invokes the read() and write() system calls of the Linux kernel. The library functions used are summarised below.

Function	Header	Purpose in the project
scanf()	<stdio.h>	Reads the scenario (sizes, vectors, matrices and requests) from standard input, which ends in a read() call
printf()	<stdio.h>	Prints the state tables, the safety trace and the decisions, which ends in a write() call
fprintf()	<stdio.h>	Reports invalid input on the standard error stream
memcpy()	<string.h>	Copies the Available vector into the Work vector at the start of the safety check

4.4 POSIX Threads / Synchronization

POSIX threads (pthreads) provide multiple flows of control inside one process, together with mutexes and semaphores for synchronization. This project does not use threads, because it simulates the decision logic in a single flow of control. Synchronization nevertheless matters in a real system: the safety check and the update of the Available vector must be carried out as one indivisible step, otherwise two simultaneous requests could each pass the safety check and together lead to an unsafe state. In a multi-threaded implementation a mutex would protect the State structure, as noted in the future scope.

4.5 Linux Commands and Utilities

Command	Use in the project
mkdir	Creating the project directory SmartResourceAllocation
cd, pwd	Moving between directories and confirming the current location

Command	Use in the project
ls	Listing the project files before and after compilation
nano	Writing banker.h, banker.c, main.c and the sample input files
gcc	Compiling and linking the source files
./banker < sample1.txt	Running the compiled program with the input file redirected to standard input

4.6 Other Tools / Libraries Used

- **Standard C library:** stdio.h (scanf, printf, fprintf) and string.h (memcpy).
- **Header-based modular design:** banker.h separates the interface of the Banker core from its implementation.
- **VS Code / GNU nano:** used for writing and editing the project source code.

CHAPTER - 5 SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The system follows a layered architecture with four layers, as shown in Figure 5.1. The user supplies a scenario and a list of requests through an input file. The interface module reads them and calls the Banker core, which reads and updates the system state and returns a decision. The decision flows back up the same path so that the user sees it on the screen.

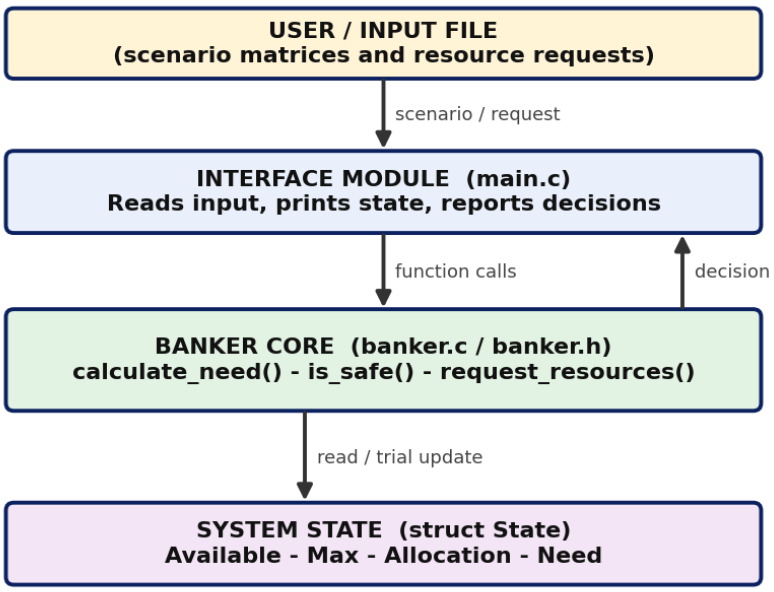


Figure 5.1: Overall architecture of the Smart Resource Allocation system

Layer	Responsibility
User / Input file	Provides n and m, the Available vector, the Max and Allocation matrices and a list of resource requests
Interface module (main.c)	Reads the input, calls calculate_need(), prints the state tables, runs the safety check, processes each request and prints the decision
Banker core (banker.c)	Computes the Need matrix, runs the Safety Algorithm (is_safe) and the Resource-Request Algorithm (request_resources) with rollback
System state (struct State)	Holds Available, Max, Allocation and Need for up to 10 processes and 10 resource types

5.2 System Workflow

The workflow for one scenario is as follows:

1. The user prepares an input file with the number of processes and resource types, the Available vector, the Max and Allocation matrices and the resource requests.
2. The program is started with ./banker < sample1.txt and the banner is displayed.
3. main.c reads the data into a State structure and calls calculate_need() to obtain Need = Max − Allocation.

4. The initial state is printed, and is_safe() is called in verbose mode: Work is set to Available and the processes are examined in turn.

5. Each process whose Need is less than or equal to Work is assumed to finish; its allocation is added to Work and it is appended to the safe sequence.

6. If all processes finish, the program prints SAFE STATE and the safe sequence; otherwise it prints UNSAFE STATE.

7. For each resource request, request_resources() checks the request against Need and Available.

8. If both checks pass, a trial allocation is made and is_safe() is run on the new state.

9. If the state is safe, the request is GRANTED; otherwise the old state is restored and the request is DENIED. A request that exceeds Need is REJECTED, and one that exceeds Available must WAIT.

5.3 Flowchart / Data Flow

Figure 5.2 shows the flow of the Safety Algorithm, and Figure 5.3 shows the flow of the Resource-Request Algorithm. The key point is that every request is evaluated on a trial state: the real state is changed permanently only when the safety check succeeds.

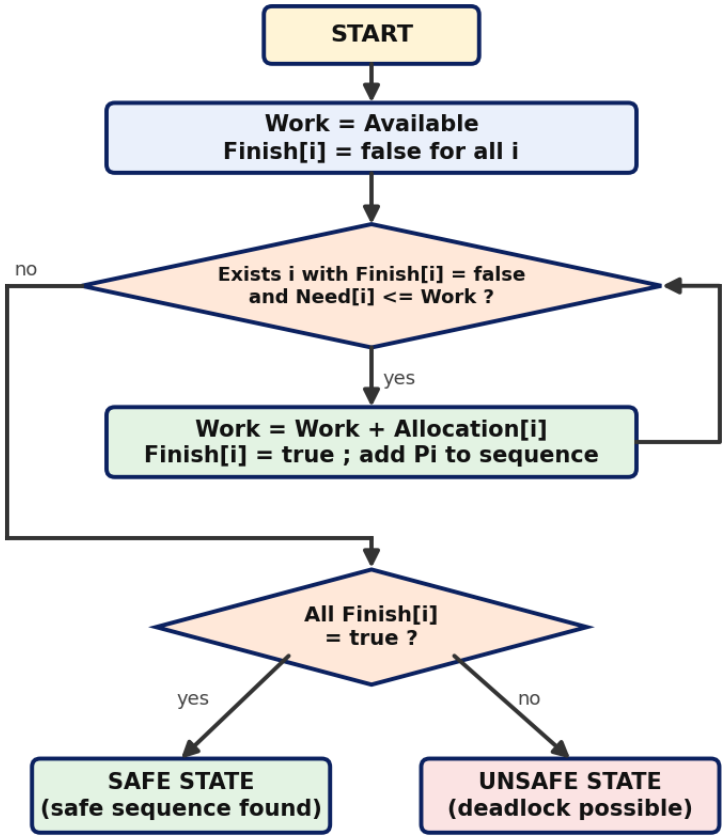


Figure 5.2: Flowchart of the Safety Algorithm

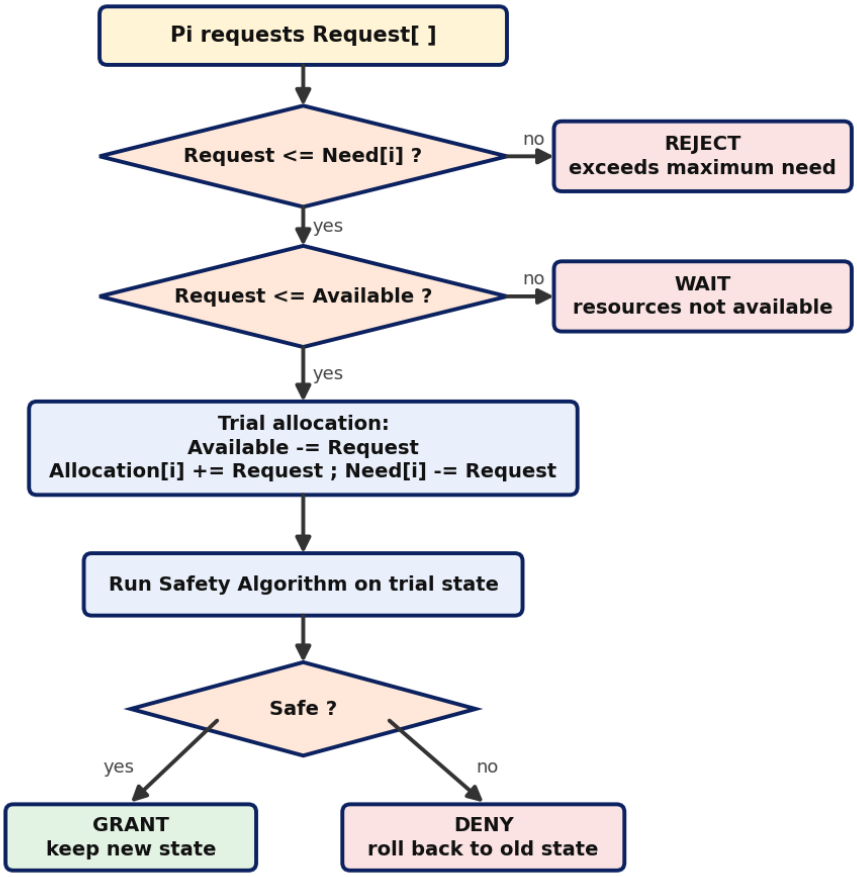


Figure 5.3: Flowchart of the Resource-Request Algorithm

5.4 Module Description

Module 1 – Interface Module (main.c)

The interface module is implemented in main.c. It reads the scenario from standard input, prints the banner, calls calculate_need(), prints the Max, Allocation and Need matrices with the Available vector, runs the safety check in verbose mode and then reads the list of requests. For each request it calls request_resources() and prints the decision.

Module 2 – Banker Core (banker.c and banker.h)

The Banker core is implemented in banker.c with its interface in banker.h. It provides three functions: calculate_need() computes Need = Max − Allocation; is_safe() carries out the Safety Algorithm and returns a safe sequence; and request_resources() carries out the Resource-Request Algorithm, using a trial allocation and a rollback. It returns one of four codes: REQ_GRANTED, REQ_EXCEEDS_NEED, REQ_MUST_WAIT or REQ_UNSAFE.

Module 3 – System State (struct State)

The State structure, defined in banker.h, stores n, m, the Available vector and the Max, Allocation and Need matrices. Because it is a plain structure without pointers, a complete copy for the trial allocation is made by a single assignment, and a rollback is a single assignment in the opposite direction.

5.5 Input and Output Design

Input

- First line: the number of processes n and the number of resource types m.
- Second line: the Available vector of m values.
- Next n lines: the Max matrix; then n lines: the Allocation matrix.
- Then the number of requests q, followed by q lines, each with a process number and m requested values.

Output

- The program banner and the initial state table showing Max, Alloc and Need for every process, with the Available vector.
- The safety trace: one line per process that can finish, showing its Need, the Work vector before it and the new Work vector after it.
- The result of the safety check (SAFE STATE or UNSAFE STATE) and the safe sequence.
- For each request, the decision: GRANTED, REJECTED, WAIT or DENIED, with the reason.

CHAPTER - 6 IMPLEMENTATION

6.1 Module-wise Implementation

The project directory is organised as follows:

```
SmartResourceAllocation/
├── banker.h      (State structure and interface)
├── banker.c      (need, safety algorithm, request algorithm)
├── main.c        (input, output and main loop)
├── sample1.txt   (five-process scenario with four requests)
└── sample2.txt   (two-process circular-wait scenario)
```

The work was divided among the team members according to the parts of the algorithm, as shown below.

Roll Number	Student Name	Individual Responsibility
2520030607	Ili Harshika	Problem analysis, input design and the data structures (Available, Max, Allocation, Need); calculation of the Need matrix.
2520030512	Gurrapu Nikshitha	Implementation of the Safety Algorithm and the search for the safe sequence.
2520030365	Angela Jose	Implementation of the Resource-Request Algorithm (trial allocation and rollback), testing and documentation.

6.2 Algorithm / Pseudocode

Algorithm for the Need Matrix

```
1.  For every process i = 0 .. n-1:
        For every resource j = 0 .. m-1:
            Need[i][j] = Max[i][j] - Allocation[i][j]
```

Algorithm for the Safety Check – is_safe()

```
1.  Work = Available
2.  Finish[i] = false for every process i; count = 0
3.  While count < n:
        found = false
        For i = 0 .. n-1:
            If Finish[i] == false AND Need[i] <= Work:
                Work = Work + Allocation[i]     /* Pi finishes, releases */
                Finish[i] = true
                Append Pi to the safe sequence; count = count + 1
                found = true
        If found == false: return UNSAFE
4.  Return SAFE (the sequence is the order in which processes were appended)
```

Algorithm for a Resource Request – request_resources(Pi, Request)

```
1.  If Request > Need[i] in any resource: return REJECTED (exceeds maximum)
2.  If Request > Available in any resource: return WAIT
3.  Save a copy of the current state
4.  Trial allocation:
        Available     = Available - Request
        Allocation[i] = Allocation[i] + Request
        Need[i]       = Need[i] - Request
5.  Run the safety check on the new state
6.  If SAFE: return GRANTED
    Else:    restore the saved state and return DENIED (unsafe)
```

6.3 Core C / C++ Implementation

The three source files of the project are given below. banker.h declares the State structure and the interface of the Banker core:

```
/* banker.h - Banker's Algorithm core interface */
#ifndef BANKER_H
#define BANKER_H

#define MAX_P 10
#define MAX_R 10

typedef struct {
    int n, m;                   /* number of processes, resource types */
    int available[MAX_R];       /* free instances of each resource     */
    int max[MAX_P][MAX_R];      /* declared maximum demand             */
    int alloc[MAX_P][MAX_R];    /* currently allocated                 */
    int need[MAX_P][MAX_R];     /* remaining need = max - alloc        */
} State;

/* return codes of request_resources() */
#define REQ_GRANTED      0
#define REQ_EXCEEDS_NEED 1
#define REQ_MUST_WAIT    2
#define REQ_UNSAFE       3

void calculate_need(State *s);
int  is_safe(const State *s, int seq[], int verbose);
int  request_resources(State *s, int pid, const int req[]);

#endif
```

banker.c contains the Need calculation, the Safety Algorithm and the Resource-Request Algorithm:

```
/* banker.c - Banker's Algorithm core: need, safety, request */
#include <stdio.h>
#include <string.h>
#include "banker.h"

static int vec_le(const int a[], const int b[], int m)
{
    for (int j = 0; j < m; j++)
        if (a[j] > b[j])
            return 0;
    return 1;
}

static void print_vec(const int v[], int m)
{
    printf("(");
    for (int j = 0; j < m; j++)
        printf("%d%s", v[j], j < m - 1 ? "," : "");
    printf(")");
}

void calculate_need(State *s)
{
    for (int i = 0; i < s->n; i++)
        for (int j = 0; j < s->m; j++)
            s->need[i][j] = s->max[i][j] - s->alloc[i][j];
}

/* Safety algorithm. Returns 1 and fills seq[] if the state is safe. */
int is_safe(const State *s, int seq[], int verbose)
{
    int work[MAX_R], finish[MAX_P] = {0};
    int count = 0;
```

```
    memcpy(work, s->available, sizeof(int) * s->m);   /* Work = Available */

    while (count < s->n) {
        int found = 0;
        for (int i = 0; i < s->n; i++) {
            if (!finish[i] && vec_le(s->need[i], work, s->m)) {
                if (verbose) {
                    printf("  P%d can finish: Need ", i);
                    print_vec(s->need[i], s->m);
                    printf(" <= Work ");
                    print_vec(work, s->m);
                }
                for (int j = 0; j < s->m; j++)
                    work[j] += s->alloc[i][j];        /* release resources */
                finish[i] = 1;
                seq[count++] = i;
                found = 1;
                if (verbose) {
                    printf(" -> Work ");
                    print_vec(work, s->m);
                    printf("\n");
                }
            }
        }
        if (!found)
            return 0;                                 /* no process can finish */
    }
    return 1;
}

/* Resource-request algorithm with trial allocation and rollback. */
int request_resources(State *s, int pid, const int req[])
{
    State saved = *s;                                 /* snapshot for rollback */
    int seq[MAX_P];

    if (!vec_le(req, s->need[pid], s->m))
        return REQ_EXCEEDS_NEED;
    if (!vec_le(req, s->available, s->m))
        return REQ_MUST_WAIT;

    for (int j = 0; j < s->m; j++) {                  /* trial allocation */
        s->available[j]  -= req[j];
        s->alloc[pid][j] += req[j];
        s->need[pid][j]  -= req[j];
    }
    if (is_safe(s, seq, 0))
        return REQ_GRANTED;

    *s = saved;                                       /* unsafe: roll back */
    return REQ_UNSAFE;
}
```

main.c contains the interface module:

```
/* main.c - user interface: reads a scenario, reports safety, serves requests */
#include <stdio.h>
#include "banker.h"

static int read_matrix(int a[][MAX_R], int n, int m)
{
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (scanf("%d", &a[i][j]) != 1)
                return 0;
    return 1;
}
```

```
static void print_state(const State *s)
{
    printf("\n  Proc   Max      Alloc    Need\n");
    for (int i = 0; i < s->n; i++) {
        printf("  P%d    ", i);
        for (int j = 0; j < s->m; j++) printf("%2d ", s->max[i][j]);
        printf("  ");
        for (int j = 0; j < s->m; j++) printf("%2d ", s->alloc[i][j]);
        printf("  ");
        for (int j = 0; j < s->m; j++) printf("%2d ", s->need[i][j]);
        printf("\n");
    }
    printf("  Available: ");
    for (int j = 0; j < s->m; j++) printf("%d ", s->available[j]);
    printf("\n");
}

static void print_sequence(const int seq[], int n)
{
    printf("  Safe sequence: ");
    for (int i = 0; i < n; i++)
        printf("P%d%s", seq[i], i < n - 1 ? " -> " : "\n");
}

int main(void)
{
    State s;
    int seq[MAX_P], q;

    if (scanf("%d %d", &s.n, &s.m) != 2 || s.n > MAX_P || s.m > MAX_R) {
        fprintf(stderr, "Invalid input\n");
        return 1;
    }
    for (int j = 0; j < s.m; j++)
        if (scanf("%d", &s.available[j]) != 1) return 1;
    if (!read_matrix(s.max, s.n, s.m) || !read_matrix(s.alloc, s.n, s.m))
        return 1;

    printf("========================================================\n");
    printf("   SMART RESOURCE ALLOCATION - BANKER'S ALGORITHM\n");
    printf("========================================================\n");

    calculate_need(&s);
    printf("[Banker] Initial state (Need = Max - Alloc)");
    print_state(&s);

    printf("\n[Banker] Running safety algorithm...\n");
    if (is_safe(&s, seq, 1)) {
        printf("[Banker] Result: SAFE STATE\n");
        print_sequence(seq, s.n);
    } else {
        printf("[Banker] Result: UNSAFE STATE - deadlock possible\n");
    }

    if (scanf("%d", &q) != 1) return 0;
    for (int k = 1; k <= q; k++) {
        int pid, req[MAX_R];
        if (scanf("%d", &pid) != 1) return 1;
        if (pid < 0 || pid >= s.n) {
            fprintf(stderr, "Invalid process number %d\n", pid);
            return 1;
        }
        for (int j = 0; j < s.m; j++)
            if (scanf("%d", &req[j]) != 1) return 1;

        printf("\n[Banker] Request %d: P%d requests (", k, pid);
        for (int j = 0; j < s.m; j++)
            printf("%d%s", req[j], j < s.m - 1 ? "," : "");
```

```
        printf(")\n");

        switch (request_resources(&s, pid, req)) {
        case REQ_GRANTED:
            printf("[Banker] Decision: GRANTED - state remains safe\n");
            is_safe(&s, seq, 0);
            print_sequence(seq, s.n);
            break;
        case REQ_EXCEEDS_NEED:
            printf("[Banker] Decision: REJECTED - "
                   "request exceeds declared maximum need\n");
            break;
        case REQ_MUST_WAIT:
            printf("[Banker] Decision: WAIT - "
                   "resources not currently available\n");
            break;
        case REQ_UNSAFE:
            printf("[Banker] Decision: DENIED - grant would lead to "
                   "an UNSAFE state (rolled back)\n");
            break;
        }
    }
    return 0;
}
```

6.4 Linux System Calls / APIs Used

Function / API	How it is used	Concept demonstrated
scanf()	Reads n, m, the vectors and the matrices; its return value is checked to detect invalid input	Input handling and validation
printf()	Prints the state tables, the safety trace and the decision for each request	Output and tracing
memcpy()	Copies the Available vector into Work at the start of the safety check	Work = Available
State copy (saved = *s)	Takes a snapshot of the whole state before the trial allocation and restores it when the state is unsafe	Trial allocation and rollback
vec_le()	Helper that compares two vectors element by element	Tests such as Need[i] <= Work

6.5 Process / Thread Implementation

The Banker's Algorithm manages processes, but this project does not create real Linux processes or threads. The processes P0 to P4 are represented by the rows of the matrices in the State structure, and the program itself is a single process with a single thread. The safe sequence is a calculated order of execution, not an order produced by the Linux scheduler. In a real operating system the same logic would be placed in the resource manager, and the rows of the matrices would correspond to the process control blocks of the running processes.

In the sample run, P1 was the first process chosen because its Need (1, 2, 2) fits within the Available vector (3, 3, 2). P3 followed because, after P1 releases its allocation (2, 0, 0), the Work vector becomes (5, 3, 2), which satisfies the Need (0, 1, 1) of P3. The remaining processes were chosen in the same way.

6.6 IPC / Synchronization Implementation

The project does not use inter-process communication such as pipes, shared memory, message queues or sockets. The only data exchange is between the interface module and the Banker core through function arguments and the shared State structure.

In a real system, however, a request must be evaluated atomically: two requests evaluated at the same time against the same Available vector could together exceed it. The usual solution is to protect the resource manager with a mutex, or to serialise all requests in a single manager process. This is noted as future work.

6.7 Memory Management Implementation

The program does not allocate memory dynamically. The State structure uses fixed-size arrays for up to 10 processes and 10 resource types and occupies 1,248 bytes on a 64-bit Linux system (312 integers). The trial allocation copies this structure into a local variable on the stack, which makes the rollback a single assignment and avoids malloc() and free(). The space needed for the matrices is O(m × n); the safety check adds only the Work vector and the Finish array.

6.8 Optional User Interface Implementation

The program provides a text-based user interface. It displays a banner, prints the Max, Allocation and Need matrices in an aligned table with the Available vector, shows each step of the safety check on one line, and labels every message with [Banker]. The final decision for each request is printed as GRANTED, REJECTED, WAIT or DENIED. No graphical interface was developed, because the goal of the project is to expose the logic of the algorithm.

CHAPTER - 7 EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

The project directory was created and the source files were written in the Ubuntu terminal. The commands used were:

```
mkdir -p SmartResourceAllocation
cd SmartResourceAllocation
nano banker.h
nano banker.c
nano main.c
nano sample1.txt
nano sample2.txt
gcc -Wall -Wextra main.c banker.c -o banker
```

The compilation completed without errors or warnings, and the executable banker was created in the project directory.

7.2 Execution Commands

```
cd ~/SmartResourceAllocation
./banker < sample1.txt
./banker < sample2.txt
```

The input file is redirected to the standard input of the program. The program prints the banner and the initial state, runs the safety check, and then processes the resource requests listed in the file.

7.3 Sample Inputs and Outputs

Sample Input 1: sample1.txt

```
5 3
3 3 2
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
4
1 1 0 2
4 3 3 0
0 0 2 0
2 0 1 0
```

The first line gives 5 processes and 3 resource types, and the second line is the Available vector (3, 3, 2). The next five lines are the Max matrix and the following five lines are the Allocation matrix. The last five lines are the number of requests (4) followed by the four requests, each starting with the process number.

Sample Output 1

```
================================================
   SMART RESOURCE ALLOCATION - BANKER'S ALGORITHM
================================================
[Banker] Initial state (Need = Max - Alloc)
  Proc   Max      Alloc    Need
  P0     7  5  3    0  1  0    7  4  3
  P1     3  2  2    2  0  0    1  2  2
  P2     9  0  2    3  0  2    6  0  0
  P3     2  2  2    2  1  1    0  1  1
  P4     4  3  3    0  0  2    4  3  1
  Available: 3 3 2

[Banker] Running safety algorithm...
  P1 can finish: Need (1,2,2) <= Work (3,3,2) -> Work (5,3,2)
  P3 can finish: Need (0,1,1) <= Work (5,3,2) -> Work (7,4,3)
  P4 can finish: Need (4,3,1) <= Work (7,4,3) -> Work (7,4,5)
  P0 can finish: Need (7,4,3) <= Work (7,4,5) -> Work (7,5,5)
  P2 can finish: Need (6,0,0) <= Work (7,5,5) -> Work (10,5,7)
[Banker] Result: SAFE STATE
  Safe sequence: P1 -> P3 -> P4 -> P0 -> P2

[Banker] Request 1: P1 requests (1,0,2)
[Banker] Decision: GRANTED - state remains safe
  Safe sequence: P1 -> P3 -> P4 -> P0 -> P2

[Banker] Request 2: P4 requests (3,3,0)
[Banker] Decision: WAIT - resources not currently available

[Banker] Request 3: P0 requests (0,2,0)
[Banker] Decision: DENIED - grant would lead to an UNSAFE state (rolled back)

[Banker] Request 4: P2 requests (0,1,0)
[Banker] Decision: REJECTED - request exceeds declared maximum need
```

The initial state is safe. The Need of P1 (1, 2, 2) fits in Work (3, 3, 2), so P1 finishes first and the Work vector grows step by step to (10, 5, 7), which is equal to the total number of instances of each resource type. The first request is granted because the state stays safe; the second must wait because the Available vector is now (2, 3, 0); the third is denied because it would lead to an unsafe state; and the fourth is rejected because it asks for more than the declared need.

Sample Input 2: sample2.txt

```
2 2
0 0
1 1
1 1
1 0
0 1
0
```

Sample Output 2

```
================================================
   SMART RESOURCE ALLOCATION - BANKER'S ALGORITHM
================================================
[Banker] Initial state (Need = Max - Alloc)
  Proc   Max      Alloc    Need
  P0     1  1    1  0    0  1
  P1     1  1    0  1    1  0
  Available: 0 0

[Banker] Running safety algorithm...
[Banker] Result: UNSAFE STATE - deadlock possible
```

In this deadlock-prone state, P0 holds the first resource and needs the second, while P1 holds the second and needs the first. With nothing available, neither Need fits in Work, so no process can finish and the state is reported as unsafe.

Test cases

ID	Function tested	Expected result	Result
TC01	Need matrix calculation	Need = Max − Allocation for every process, for example P0 = (7, 4, 3) and P4 = (4, 3, 1)	PASS
TC02	Safety Algorithm	Safe sequence P1 → P3 → P4 → P0 → P2 is found for Available (3, 3, 2)	PASS
TC03	Valid request	P1 requests (1, 0, 2): granted and the state remains safe	PASS
TC04	Request exceeding Available	P4 requests (3, 3, 0) when Available is (2, 3, 0): P4 must wait	PASS
TC05	Request leading to an unsafe state	P0 requests (0, 2, 0): denied and the state is rolled back	PASS
TC06	Request exceeding Need	P2 requests (0, 1, 0) but its Need in that resource is 0: rejected	PASS
TC07	Deadlock-prone state	Two-process circular wait is reported as an unsafe state	PASS

7.4 Screenshots of Execution

Figure 7.1 shows the compilation with GCC and the start of the program for Sample Input 1, with the banner, the initial state table and the step-by-step safety check that ends with the safe sequence.

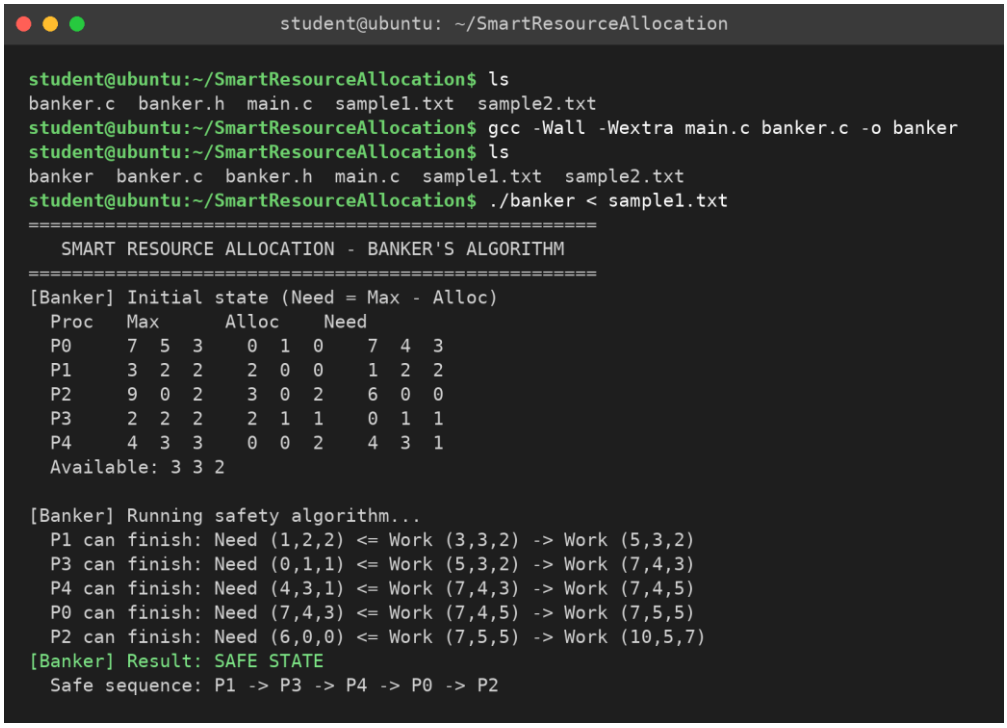


Figure 7.1: Compilation and safety check for Sample Input 1

Figure 7.2 shows the decisions made for the four resource requests of Sample Input 1: one request granted, one made to wait, one denied as unsafe and one rejected.

```
student@ubuntu: ~/SmartResourceAllocation

[Banker] Request 1: P1 requests (1,0,2)
[Banker] Decision: GRANTED - state remains safe
  Safe sequence: P1 -> P3 -> P4 -> P0 -> P2

[Banker] Request 2: P4 requests (3,3,0)
[Banker] Decision: WAIT - resources not currently available

[Banker] Request 3: P0 requests (0,2,0)
[Banker] Decision: DENIED - grant would lead to an UNSAFE state (rolled back)

[Banker] Request 4: P2 requests (0,1,0)
[Banker] Decision: REJECTED - request exceeds declared maximum need
```

Figure 7.2: Decisions for the four resource requests

Figure 7.3 shows Sample Input 2, in which the circular wait of two processes is reported as an unsafe state.

```
student@ubuntu: ~/SmartResourceAllocation

student@ubuntu:~/SmartResourceAllocation$ ./banker < sample2.txt
================================================
   SMART RESOURCE ALLOCATION - BANKER'S ALGORITHM
================================================
[Banker] Initial state (Need = Max - Alloc)
  Proc   Max      Alloc    Need
  P0     1  1    1  0    0  1
  P1     1  1    0  1    1  0
  Available: 0 0

[Banker] Running safety algorithm...
[Banker] Result: UNSAFE STATE - deadlock possible
```

Figure 7.3: Deadlock-prone state reported as unsafe

Figure 7.4 plots the Work vector after each process finishes in the safe sequence of Sample Input 1. The number of available instances of every resource type never decreases along the sequence, and the final vector (10, 5, 7) equals the total number of instances in the system.

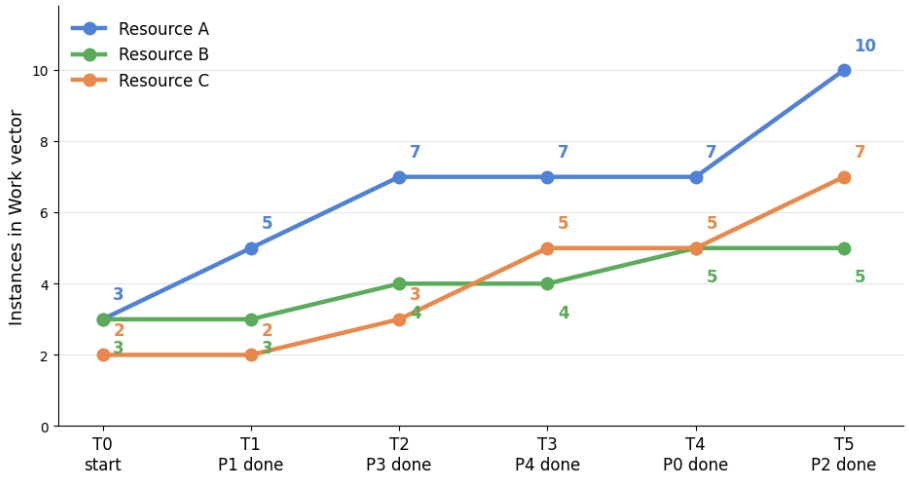


Figure 7.4: Growth of the Work vector along the safe sequence

7.5 Demonstration of OS Concepts

OS concept	Where it is visible in the output
Resource allocation state	The Max, Alloc and Need table and the Available vector printed at the start of the output.
Need matrix	The Need column equals Max − Alloc: P0 shows (7, 4, 3) and P1 shows (1, 2, 2).
Safe state and safe sequence	"Result: SAFE STATE" followed by P1 → P3 → P4 → P0 → P2.
Resource release (Work vector)	Each trace line shows Work growing as a process finishes: (3, 3, 2), (5, 3, 2), (7, 4, 3), (7, 4, 5), (7, 5, 5) and finally (10, 5, 7).
Deadlock avoidance	The request of P0 for (0, 2, 0) is denied although the resources are available, because the new state would be unsafe.
Circular wait / unsafe state	In Sample Input 2 neither process can finish with Available (0, 0), so the state is reported as UNSAFE.

CHAPTER - 8 CONCLUSION

8.1 Summary of the Project

Smart Resource Allocation Using Banker's Algorithm was designed, implemented and tested successfully. The system uses the Available vector and the Max, Allocation and Need matrices to describe the state of the resources. For every request, a trial allocation is made, the Safety Algorithm searches for a safe sequence, and the request is granted only if such a sequence exists; otherwise the old state is restored. In this way a granted request never moves the system into an unsafe state, and deadlock caused by circular wait is avoided.

8.2 Objectives Achieved

Objective	Concept used	Status
Represent the resource-allocation state of the system	Available, Max, Allocation, Need (struct State)	Achieved
Compute the Need matrix from Max and Allocation	Need = Max − Allocation	Achieved
Implement the Safety Algorithm and find a safe sequence	Work, Finish, safe sequence	Achieved
Grant a request only when the resulting state is safe	Trial allocation and rollback	Achieved
Demonstrate and verify the system with test cases	sample1.txt, sample2.txt, TC01–TC07	Achieved

8.3 Results Achieved

- The system was compiled with GCC (-Wall -Wextra) without errors or warnings and executed on Ubuntu.
- For the standard example with Available (3, 3, 2), the system found the safe sequence P1 → P3 → P4 → P0 → P2, and the Work vector ended at (10, 5, 7), equal to the total number of instances.
- Safe requests were granted, requests beyond the declared need or the available resources were rejected or made to wait, and a request leading to an unsafe state was denied and rolled back.
- A circular-wait scenario was correctly reported as an unsafe state.
- All seven test cases (TC01–TC07) passed.

8.4 Skills and Learning Outcomes

- Practical understanding of deadlock, safe and unsafe states, and the Banker's Algorithm.
- Experience in implementing matrix-based algorithms in C with header files and separate compilation.
- Use of the Linux terminal, nano and GCC, and skills in testing, documenting and presenting a systems project as a team.

Future scope. The system can be extended with dynamic arrival of processes, a deadlock-detection and recovery module, a multi-threaded version with a mutex around the State structure, a graphical interface, and integration with real Linux processes.

CHAPTER - 9 REFERENCES

9.1 Books

[1] A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, Wiley.

[2] A. S. Tanenbaum and H. Bos, Modern Operating Systems, Pearson.

[3] W. Stallings, Operating Systems: Internals and Design Principles, Pearson.

[4] M. Kerrisk, The Linux Programming Interface, No Starch Press, 2010.

[5] B. W. Kernighan and D. M. Ritchie, The C Programming Language, Prentice Hall.

9.2 Research Papers

[1] E. W. Dijkstra, “Een algorithme ter voorkoming van de dodelijke omarming,” EWD 108, 1965.

[2] E. G. Coffman, M. J. Elphick and A. Shoshani, “System Deadlocks,” ACM Computing Surveys, vol. 3, no. 2, 1971.

[3] A. N. Habermann, “Prevention of System Deadlocks,” Communications of the ACM, vol. 12, no. 7, 1969.

[4] R. C. Holt, “Some Deadlock Properties of Computer Systems,” ACM Computing Surveys, vol. 4, no. 3, 1972.

9.3 Linux / GNU Documentation

[1] Linux manual pages: scanf(3), printf(3), memcpy(3).

[2] The GNU C Library Reference Manual.

[3] GCC, the GNU Compiler Collection – Online Documentation.

9.4 Online Resources

[1] Linux man-pages project: https://man7.org/linux/man-pages/

[2] GNU Project documentation: https://www.gnu.org/software/libc/manual/